Test answers-

Plan –

On the 4 test answers

I can generate those right now — no setup needed. I'd write them at four levels:

- **Strong PASS** — hits all 3 concepts clearly, uses enterprise language naturally
- **Weak PASS** — gets all 3 but the SOX audit trail piece is thin/implicit
- **PARTIAL** — solid on hooks but misses the audit trail entirely, or gets audit trail but thinks better prompting is part of the fix
- **FAIL** — developer mindset, focuses on prompt wording or adding logging calls

We can analyze what the rubric *should* produce for each, then once you're live, run them through the actual grader and see if Claude's output matches expectations. That's your real test of whether the framework holds up.

Test Questions

Here are four answers written as realistic exam responses — the kind of range you'd actually see from people with varying backgrounds.

Answer 1 — Strong PASS

The core architectural failure is that this system relies entirely on system prompt instructions to enforce a financial control. That is not a control — it is a suggestion. Language model behavior is probabilistic. In a SOX-regulated environment, any control with a non-zero failure rate is an audit finding waiting to happen. The $50,000 approval threshold is a material internal control under SOX Section 302/404. It cannot live in a prompt.

The fix requires PostToolUse hooks on the process_payment tool. The hook intercepts every call to that tool before execution, reads the invoice amount, checks whether a documented manager approval exists in the authorization record, and blocks execution if the threshold is exceeded without it. The agent's instructions become irrelevant to enforcement — the hook either permits or denies the call regardless of what the prompt says.

The second issue the Finance Director is almost certainly flagging is the audit trail. SOX doesn't just require that controls exist — it requires documented evidence that they fired on specific

transactions. A write_audit_log tool call that the agent decides to make is not sufficient. The hook must generate its own tamper-evident log entry: timestamped, recording the tool call parameters, the authorization check result, and the permit/block decision. This record needs to exist independent of the agent's behavior, because the agent's behavior is exactly what auditors are verifying.

Putting process_payment in the tools list with no hook is the functional equivalent of telling your AP clerk "please remember to get approval for large payments" and calling that your internal control.

Answer 2 — Weak PASS

The problem is that the approval requirement for payments over $50,000 is written into the system prompt as an instruction, not enforced programmatically. LLMs don't guarantee that they follow instructions 100% of the time — there's always some probability of the instruction being ignored, misinterpreted, or bypassed in an edge case. For a financial control that needs to be reliable, that's not acceptable.

The right approach is to implement hooks in the Agent SDK that intercept the process_payment tool call. Before the payment executes, the hook should check whether manager approval has been documented for that invoice. If not, the hook blocks the call and routes it to the approval workflow. This makes the control deterministic — it doesn't matter what the agent was told to do, the hook enforces the rule at the infrastructure level.

For SOX compliance you also need logging. Every time the hook fires, it should record what happened — which invoice, what amount, whether it was approved or blocked, and when. This gives auditors the evidence they need to verify the control is working. The system currently relies on the agent calling write_audit_log itself, which isn't reliable enough because it's still just an instruction.

Answer 3 — PARTIAL

The architecture has two problems. First, the system prompt instructions are the only thing enforcing the approval requirement, and they're not specific enough. The prompt says "you MUST require manager approval" but doesn't define what counts as approval, how to verify it, or what to do if the manager doesn't respond. The agent could interpret this in different ways.

Second, and more importantly, there are no hooks configured. The Agent SDK has a hook system specifically designed for this kind of workflow enforcement. A pre-execution hook on the

process_payment tool would intercept the call before it runs, verify that the invoice amount has been approved, and block it if not. This is much more reliable than relying on the agent to make the right decision based on prompt instructions alone.

To fix this: implement hooks on process_payment, strengthen the prompt language to be more explicit about the approval verification steps, and make sure the notify_manager tool is called before process_payment in any workflow involving amounts over the threshold. The Finance Director is right that this needs to be tightened up before it can go to a SOX audit.

Answer 4 — FAIL

The Finance Director's concern is valid. Looking at the code, there are several issues that would cause problems in a SOX audit.

The system prompt mentions the $50,000 threshold but the process_payment tool has no amount limit — the comment even says "Can execute payments up to any amount." This is a direct contradiction. The tool should have validation built in to reject payments over the threshold unless an approval flag is set.

The audit log is also a concern. The agent is instructed to "always document your actions in the audit log" but this is vague. For SOX compliance, audit entries need to include specific fields — invoice ID, amount, approver name, timestamp, and approval date. The current instruction doesn't specify any of this.

I would recommend rewriting the system prompt to be much more specific: require the agent to call notify_manager and wait for a confirmation response before calling process_payment on any invoice over $50,000. Add validation parameters to the process_payment tool itself. And define a structured format for audit log entries that captures all the fields an auditor would need to see.

What I expect the grader to do with these:

Answer	Expected	Reasoning
1	PASS	All 3 concepts, strong SOX framing, "tamper-evident" language
2	PASS	All 3 present — audit trail is thin but the concept is there

Answer	Expected	Reasoning
3	PARTIAL	Hooks + enforcement gap are solid; audit trail is missing; the "strengthen the prompt" line is actively wrong
4	FAIL	Prompt rewriting and tool validation — never grasps why the prompt itself is the problem

The interesting one to watch is Answer 2 — if the grader is lenient it's a PASS, if strict it might land PARTIAL. That will tell you something real about the rubric calibration.